

Práctica 3 - Búsqueda Binaria

Abigail Sánchez

1. Introducción

El rendimiento de los algoritmos de búsqueda incide directamente en el rendimiento de aplicaciones que manejan grandes volúmenes de datos. En esta práctica, se estudia el funcionamiento del algoritmo de búsqueda binaria -tanto su versión iterativa como la recursiva, enfocándonos más en la segunda-, para la cual primero se determina la complejidad validando el resultado con el Teorema Maestro. Finalmente complementamos el análisis con resultados experimentales corroborando que tiene el mismo orden de complejidad $\Theta(\log(n))$, cumpliendo así el principio de invarianza.

2. Marco Teórico

2.1. Búsqueda Binaria Iterativa

De manera iterativa, el algoritmo de búsqueda binaria se puede implementar mediante el siguiente pseudocódigo.

Entradas:

- A : un arreglo ordenado.
- n : tamaño de A .
- x : elemento que se está buscando.

```
busquedaBinaria(A, n, x):  
(1)   m0 = 0; m2 = n-1;  
(2)   while(m2-m0 > 2):  
(3)       m1 = m0+(m2-m0)/2;  
(4)       if(x<A[m1]):  
(5)           m2 = m1-1;  
(6)       else if(x==A[m1]):  
(7)           return m1;  
(8)       else m0 = m1+1;  
(9)   return -1;
```

Ahora comencemos a analizar cada línea de código. Para ello, denotaremos con $T_i(n)$ al número de operaciones elementadas realizadas al ejecutar la i -ésima línea de código.

- $T_1(n) = 3$ (2 asignaciones, 1 resta)
- $T_3(n) = 4$ (1 asignación, 1 suma, 1 resta, 1 división)
- $T_4(n) = 3$ (1 salto, 1 acceso, 1 comparación)
- $T_5(n) = 2$ (1 asignación, 1 resta)
- $T_6(n) = 3$ (1 salto, 1 comparación, 1 acceso)
- $T_7(n) = 2$ (1 asignación, 1 retorno)
- $T_8(n) = 2$ (1 asignación, 1 suma)
- $T_9(n) = 2$ (1 asignación, 1 retorno)

Ahora sea $T_2(n)$ el número de operaciones elementales se realizan al ejecutar el bloque de código subordinado al ciclo while en la segunda línea. Entonces, en el peor caso cuando x no se encuentra en A , en cada iteración se alcanza el segundo condicional if y se realizan

$$T_{\text{interior}}(n) = T_3(n) + T_4(n) + T_6(n) + T_8(n) = 4 + 3 + 3 + 2 = 12$$

Si el ciclo while hace M iteraciones, entonces en cada una de ellas se realizan $T_{\text{interior}}(n) + 2 = 14$ operaciones elementales. Las 2 operaciones elementales adicionales corresponden a la comparación y a la resta en la condición del ciclo while.

Por último, al entrar al ciclo while se hace un salto y al concluir con las M iteraciones se hace una última comparación, la cual al ser falsa, provoca que el ciclo termine sin hacer las operaciones en el interior. Esto suma 3 operaciones elementales al total de tal manera que

$$T_2(n) = 14M + 3$$

Y por lo tanto

$$T(n) = T_1(n) + T_2(n) + T_9(n) = 14M + 8$$

Falta determinar el número de iteraciones que hace el ciclo while. Para esto consideremos que en la última iteración tenemos 1 elemento sin revisar, mientras que en la penúltima tenemos 2 elementos y en la anterior 4 elementos. Podemos seguir duplicando el número de elementos al regresar en las iteraciones hasta alcanzar el tamaño original n del arreglo A de tal manera que M es el mínimo entero que satisface que

$$n \leq 2^M$$

o equivalentemente

$$M \geq \log(n)$$

Entonces basta con tomar $M = \lceil \log(n) \rceil$ con lo que obtenemos el resultado final:

$$T(n) = 14\lceil \log(n) \rceil + 8 \in \Theta(\log(n)) \quad (1)$$

2.2. Búsqueda Binaria Recursiva

Una implementación recursiva del algoritmo de búsqueda binaria está dada por el pseudocódigo siguiente.

Entradas:

- A : un arreglo ordenado.
- $m0$: índice inicial de la sección de A en la cual se realiza la búsqueda.
- $m2$: índice final de la sección de A en la cual se realiza la búsqueda.
- x : elemento que se está buscando.

```
busquedaBinariaR(A, m0, m2, x):  
(1)   if(m0<m2):  
(2)       m1=m0+(m2-m0)/2;  
(3)       if(x<A[m1]):  
(4)           return busquedaBinariaR(A, m0, m1-1, x);  
(5)       else if(x>A[m1]):  
(6)           return busquedaBinariaR(A, m1+1, m2, x);  
(7)       else return m1;  
(8)       return (x==A[m0]) ? m0 : -1;
```

- $T_1(n) = 2$ (1 salto, 1 comparación)
- $T_2(n) = 4$ (1 asignación, 1 suma, 1 resta, 1 división)
- $T_3(n) = 3$ (1 salto, 1 comparación, 1 acceso)
- $T_5(n) = 3$ (1 salto, 1 comparación, 1 acceso)
- $T_7(n) = 2$ (1 asignación, 1 retorno)
- $T_8(n) = 5$ (1 comparación, 1 acceso, 1 salto, 1 asignación, 1 retorno)

Por último, tenemos que en las líneas 4 y 6 se llama a la misma función búsqueda binaria pero sustituyendo a uno de los extremos con el antecesor o el sucesor de $m1$. Como $m1$ es el punto medio entre los extremos $m0$ y $m2$, entonces el tamaño del arreglo que se le pasa a la búsqueda binaria en el llamado recursivo es $\frac{n}{2}$. Esto quiere decir que:

- $T_4(n) = T\left(\frac{n}{2}\right) + 8$ (recursión, 5 asignaciones, 1 llamado, 1 resta, 1 retorno)
- $T_6(n) = T\left(\frac{n}{2}\right) + 8$ (recursión, 5 asignaciones, 1 llamado, 1 suma, 1 retorno)

Para analizar el algoritmo recursivo, primero debemos identificar un caso base. Si $n = 1$, entonces $m0 = m2$, por lo que el algoritmo termina de inmediato revisando solamente la condición del if una sola vez y retornando $m0$ o -1 dependiendo de si el único elemento del arreglo es igual a x . En cualquier caso, tenemos que

$$T(1) = T_1(1) + T_8(1) = 2 + 5 = 7$$

Por otro lado, para el peor caso con n arbitraria tenemos que

$$T(n) = T_1(n) + T_2(n) + T_3(n) + T_5(n) + T_6(n) = T\left(\frac{n}{2}\right) + 20$$

Entonces tenemos la ecuación en recurrencias

$$T(n) = \begin{cases} 7, & n = 1 \\ T\left(\frac{n}{2}\right) + 20, & n > 1 \end{cases} \quad (2)$$

Para resolver esta ecuación proponemos el cambio de variable $n = 2^k$ y $H(k) = T(2^k)$ de tal forma que

$$H(k) = H(k - 1) + 20$$

Esta ecuación tiene polinomio característico

$$(x - 1)^2 = 0$$

De donde se concluye que la solución tiene la forma

$$H(k) = c_1 \cdot k + c_2$$

o bien

$$T(n) = c_1 \log(n) + c_2$$

Para determinar las constantes c_1 y c_2 podemos usar la condición inicial

$$T(1) = c_1 \log(1) + c_2 = c_2 = 7$$

Además, usando la ecuación 2 obtenemos una segunda condición inicial

$$T(2) = T(1) + 7 = 27$$

Con lo cual concluimos que

$$T(2) = c_1 \log(2) + 7 = c_1 + 7 = 27 \Rightarrow c_1 = 20$$

Y por lo tanto el número de operaciones elementales que realiza el algoritmo es

$$T(n) = 20 \log(n) + 7 \in \Theta(\log(n)) \quad (3)$$

Observemos que los tiempos de ejecución de ambas implementaciones dados por 1 y 3 tienen el mismo orden logarítmico $\Theta(\log(n))$ como se esperaría por el principio de invarianza.

2.3. Teorema Maestro

Regresando a la ecuación 2, es decir a que para el algoritmo recursivo se cumple que $T(n) = T\left(\frac{n}{2}\right) + 20$, identificamos a los componentes del teorema maestro

$$a = 1, \quad b = 2, \quad f(n) = 20$$

El exponente crítico entonces es

$$c = \log_b(a) = \log(1) = 0$$

Por lo que observamos que

$$f(n) = 20 \in \Theta(1) = \Theta(n^0 \log^0(n)) = \Theta(n^c \log^k(n))$$

con $k = 0$.

Esto quiere decir que se cumplen las hipótesis del Teorema Maestro y que por lo tanto

$$T(n) \in \Theta(n^c \log^{k+1}(n)) = \Theta(n^0 \log^1(n)) = \Theta(\log(n))$$

ratificando el resultado que habíamos obtenido al resolver de manera exacta las ecuaciones 1 y 3.

3. Resultados

La gráfica muestra los tiempos promedio medidos para el algoritmo, en función del tamaño del arreglo n . Los puntos azules son los valores resultantes durante la ejecución, mientras que la línea roja es el ajuste obtenido mediante regresión logarítmica; este nos indica que el tiempo de ejecución crece de forma logarítmica con respecto al tamaño del arreglo y solo para terminar de confirmar, el coeficiente de determinación R^2 nos muestra que el ajuste entre los datos del modelo logarítmico y los resultados, es casi 1, lo cual es consistente con el análisis teórico de la complejidad del algoritmo siendo $\Theta(\log(n))$.

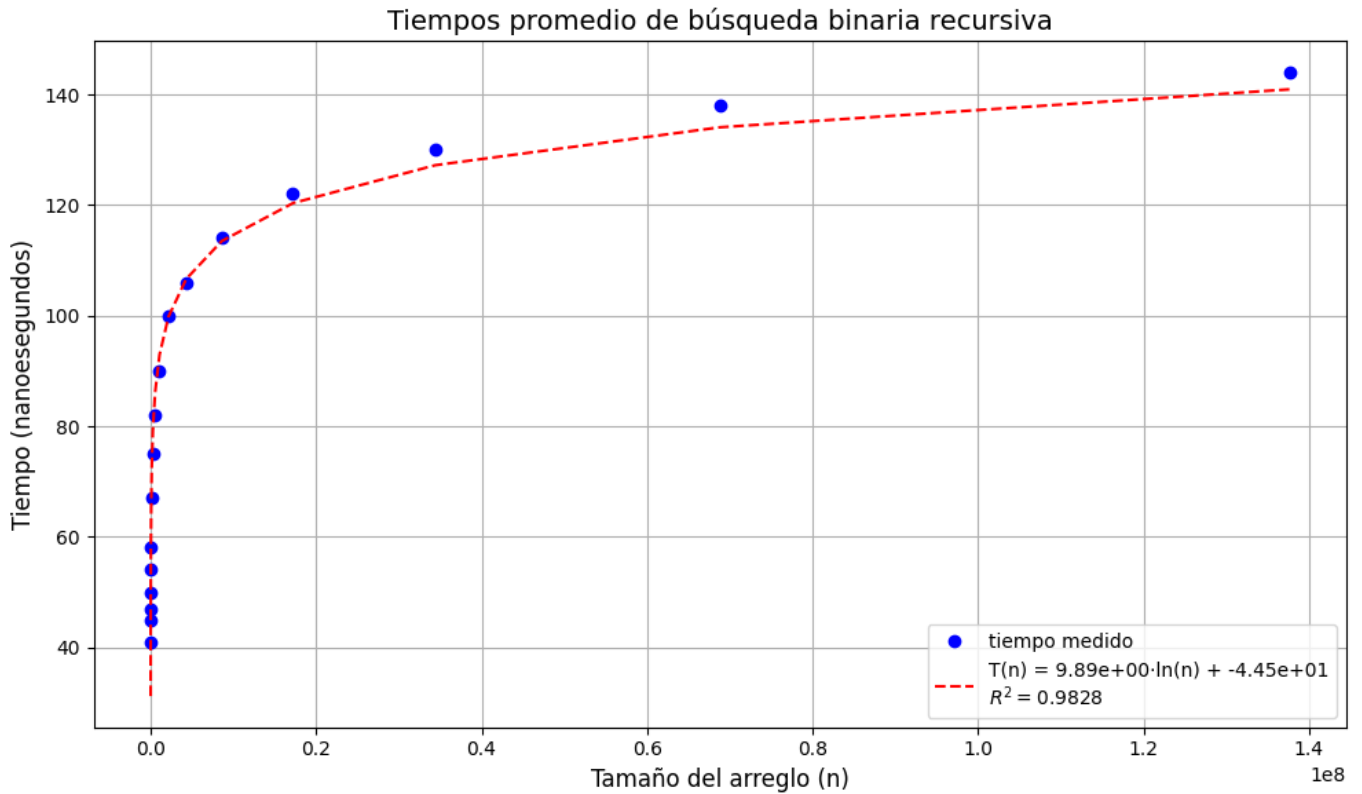


Figura 1: Curva de mejor ajuste logarítmico.

4. Conclusiones

Los resultados corroboran la hipótesis que formulamos al analizar de manera teórica el algoritmo, ya que los datos se ajustan con alta significancia estadística a una curva logarítmica.

Esto significa que el algoritmo se ejecuta tan velozmente que es difícil para la computadora medir el tiempo de ejecución. Por ello, debimos ejecutar el algoritmo millones de veces y medir el tiempo que se tomó en concluir con el conjunto de todas ellas y luego dividir entre el número de repeticiones para aislar el tiempo que toma ejecutarlo una sola vez. Esta técnica tiene la ventaja adicional de que, al considerar más datos, reduce la variación de los mismos por factores externos (ruido).

Otro problema que se presenta al trabajar con un algoritmo tan eficiente es que espaciar los tamaños de los arreglos linealmente (e.g. utilizando incrementos de 50,000 en 50,000) no permite observar adecuadamente el comportamiento al realizar la regresión debido a que al ser logarítmico pero redondeado al entero más cercano (ya que no existen medias llamadas a la función), el número de iteraciones es el mismo hasta que no se alcanza la siguiente potencia de 2. Por ejemplo, se alcanza el mismo nivel de recursión si $n = 2,000$ que si $n = 3,000$. Esto genera que los datos se acomoden en escalones si se usan incrementos constantes como $i+ = 50,000$. Para solucionar este problema, se usaron incrementos multiplicativos $i* = 2$ que aseguran que en cada iteración superemos a la siguiente potencia de 2.

En conjunto, los resultados validan el modelo propuesto y demuestran que la búsqueda binaria es eficaz incluso a gran escala.